AI VIET NAM
@aivietnam.edu.vn

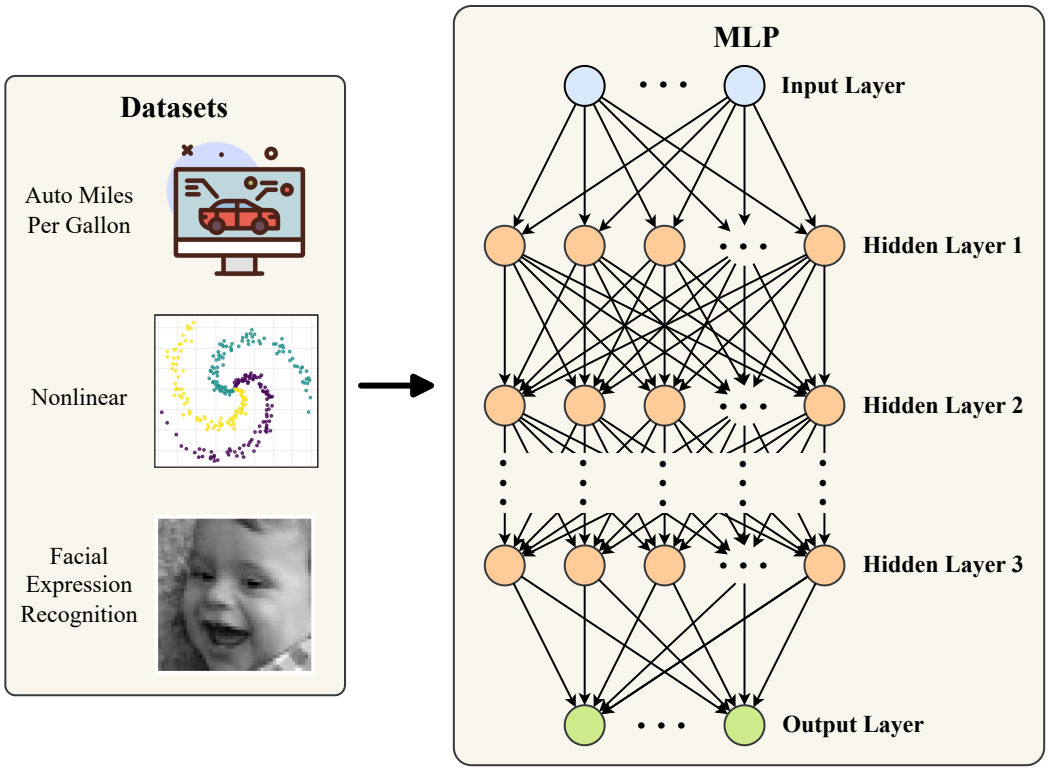
AI VIET NAM – AI COURSE 2026

Exercise: Multi-layer Perceptron

Dương Đình Thắng Nguyễn Anh Khôi Đinh Quang Vinh

I. Giới thiệu

Multi-layer Perceptron (MLP) là kiến trúc neural network cơ bản và quan trọng trong lĩnh vực deep learning. MLP được cấu tạo từ ba thành phần chính, bao gồm input layer nhận dữ liệu đặc trưng, hidden layers thực hiện biến đổi phi tuyến thông qua activation functions, và output layer tạo ra kết quả dự đoán cuối cùng. Khả năng học các mối quan hệ phi tuyến phức tạp giữa đầu vào và đầu ra khiến MLP trở thành công cụ mạnh mẽ cho đa dạng bài toán như classification, regression và pattern recognition.



Hình 1: Kiến trúc tổng quát của Multi-layer Perceptron (MLP) với các lớp fully connected. Mỗi neuron ở lớp hiện tại kết nối với tất cả neuron ở lớp trước đó.

Mục lục

I. Giới thiệu **1**

II. Thực hành **5**

II.1. Bài tập 1: Dự đoán hiệu suất xe hơi 5

II.2. Bài tập 2: Phân loại với dữ liệu phi tuyến 8

II.3. Bài tập 3: Phân loại cảm xúc trên ảnh 12

Phụ lục **17**

Trong bài tập này, chúng ta sẽ thực hành cách triển khai và huấn luyện mạng MLP bằng thư viện PyTorch thông qua ba bài tập thực hành với độ phức tạp tăng dần. Nhiệm vụ của các bạn là sẽ hoàn thiện các file code Hint trong phần II.3.3. để triển khai thành công chương trình. Nội dung chính của các câu bài tập như sau:

Tổng quan các bài tập lập trình

Bài	Mục tiêu	Biểu diễn/Đầu vào	Hàm mục tiêu/Đánh giá
1	Xây dựng MLP với PyTorch cho bài toán hồi quy Auto MPG, huấn luyện và đánh giá.	Dữ liệu CSV với 9 đặc trưng xe hơi đã chuẩn hóa, $X \in \mathbb{R}^{n\times 9}$, $y \in \mathbb{R}^n$ (MPG liên tục).	Mean Squared Error (MSE), theo dõi loss và R^2 score, đánh giá trên validation và test.
2	Xây dựng MLP cho phân loại dữ liệu phi tuyến 3 lớp trong không gian 2D.	Dữ liệu .npy với 2 đặc trưng đã chuẩn hóa, $X \in \mathbb{R}^{300\times 2}$, $y \in \{0,1,2\}^{300}$.	Cross Entropy Loss, accuracy đa lớp, trực quan hóa đường hội tụ train/validation loss và accuracy.
3	Xây dựng MLP sâu với PyTorch cho phân loại cảm xúc khuôn mặt FER-2013.	Ảnh grayscale 128×128 flatten → 16384 chiều, chuẩn hóa về [-1,1]; $X \in \mathbb{R}^{n\times 16384}$, $y \in \{0,\ldots,6\}^n$ (7 cảm xúc).	Cross Entropy Loss, accuracy đa lớp trên validation/test, kiến trúc pyramid 3 lớp ẩn (256→128→64), trực quan hóa hội tụ.

Mặc dù có sự khác biệt về bối cảnh ứng dụng (hồi quy, phân loại phi tuyến, phân loại ảnh), việc cài đặt và huấn luyện mô hình MLP với PyTorch đều tuân theo quy trình chung như sau:

1. **Nhập thư viện và thiết lập môi trường:** Import PyTorch, NumPy, pandas, scikit-learn, matplotlib; cài đặt seed cố định cho reproducibility; thiết lập device (GPU/CPU).
2. **Chuẩn bị dữ liệu:** Tải dữ liệu (CSV/NPY/ZIP), tách `X` và `y`, chia train/validation/test theo tỉ lệ 7:2:1, chuẩn hóa đặc trưng bằng StandardScaler (fit trên train, transform trên validation/test), chuyển đổi sang PyTorch tensor với dtype phù hợp.
3. **Xây dựng DataLoader:** Tạo class `CustomDataset` kế thừa `torch.utils.data.Dataset`, triển khai `__len__` và `__getitem__`, khởi tạo DataLoader với batch size phù hợp (train có shuffle=True, validation/test có shuffle=False).
4. **Định nghĩa kiến trúc MLP:** Xây dựng class kế thừa `nn.Module`, định nghĩa các lớp Linear và activation (ReLU) trong `__init__`, triển khai forward pass với số lớp ẩn và hidden dimensions phù hợp với độ phức tạp bài toán.

5. **Khai báo loss và optimizer:** Chọn hàm loss phù hợp (`nn.MSELoss` cho regression, và classification là `nn.CrossEntropyLoss`), khởi tạo optimizer (`torch.optim.SGD`) với learning rate thích hợp.

6. **Huấn luyện:** Chuyển sang `model.train()`, duyệt qua train_loader, thực hiện forward pass, tính loss, backward propagation (`loss.backward()`), cập nhật weights (`optimizer.step()`); chuyển sang `model.eval()` với `torch.no_grad()` để đánh giá validation, ghi lại loss/metrics cho cả train và validation.

7. **Đánh giá và trực quan:** Tính metrics cuối cùng (R^2 score cho regression, accuracy cho classification) trên test set, vẽ biểu đồ loss/metrics theo epoch để phân tích hội tụ và phát hiện overfitting.

Với quy trình chung đã trình bày, phần tiếp theo sẽ hướng dẫn chi tiết cách triển khai ba bài tập thực hành với PyTorch.

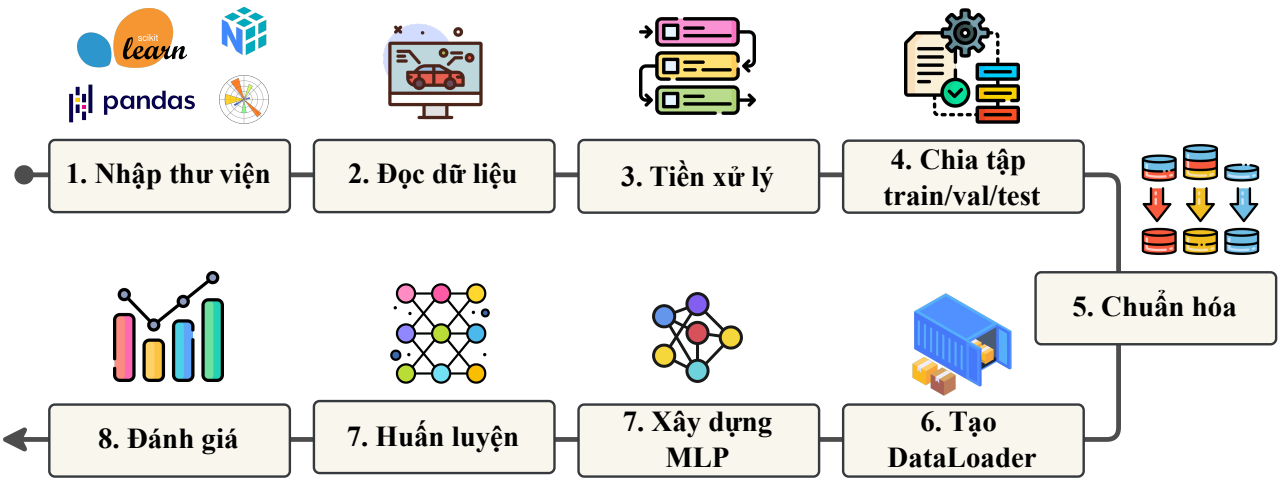
II. Thực hành

II.1. Bài tập 1: Dự đoán hiệu suất xe hơi

II.1.1. Phát biểu bài toán

Nội dung

Xây dựng mạng **Multi-layer Perceptron (MLP)** với PyTorch cho bài toán **Auto MPG Prediction**. Thực hiện tiền xử lý, tách tập train/validation/test, xây dựng DataLoader, huấn luyện bằng SGD với hàm loss MSE, theo dõi loss/R^2 score và đánh giá cuối cùng trên tập test.



Hình 2: Quy trình xây dựng MLP cho bài tập dự đoán hiệu suất xe hơi.

Trong bài tập này, ta làm quen với việc xây dựng mạng MLP sử dụng PyTorch thông qua bài toán hồi quy (regression) dự đoán mức tiêu thụ nhiên liệu (MPG - Miles Per Gallon) của xe hơi dựa trên các đặc trưng kỹ thuật. Mô hình MLP gồm 2 lớp ẩn với hàm kích hoạt ReLU, được tối ưu hóa bằng thuật toán Stochastic Gradient Descent với hàm loss Mean Squared Error. Quá trình huấn luyện được giám sát qua hai chỉ số: MSE loss và R^2 score (Coefficient of determination).

II.1.2. Mục tiêu của bài tập

- Thành thạo việc sử dụng PyTorch để xây dựng mạng MLP với các thành phần: `nn.Module`, `nn.Linear`, `F.relu`.
- Thiết lập pipeline hoàn chỉnh: đọc dữ liệu CSV, tách train/validation/test theo tỉ lệ 7:2:1, chuẩn hóa đặc trưng bằng StandardScaler.
- Xây dựng PyTorch Dataset và DataLoader để quản lý dữ liệu hiệu quả với mini-batch.
- Cài đặt seed cố định cho tính tái lập và tự động chọn thiết bị GPU/CPU.

- Huấn luyện mô hình với vòng lặp epoch, theo dõi đường hội tụ loss và R^2 score trên cả train và validation.
- Đánh giá khả năng tổng quát hóa của mô hình thông qua R^2 score trên tập test.

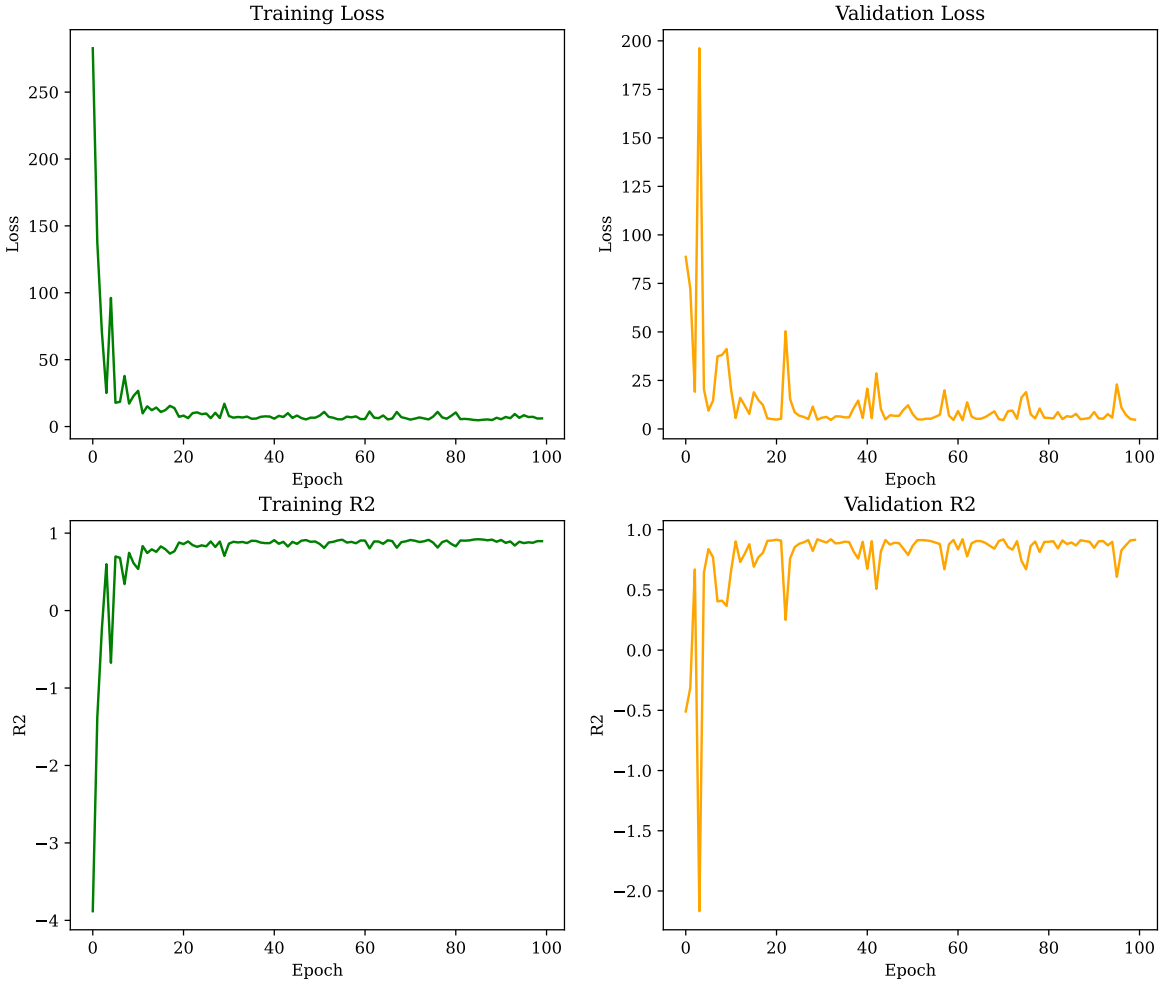
II.1.3. Yêu cầu

Dựa vào hướng dẫn chi tiết dưới đây, hãy hoàn thiện pipeline huấn luyện mô hình MLP cho bài toán Auto MPG:

- **Chuẩn bị môi trường:** Tải bộ dữ liệu Auto_MPG_data.csv, import các thư viện cần thiết (PyTorch, NumPy, pandas, scikit-learn), cài đặt seed cố định cho NumPy và PyTorch, thiết lập device (GPU/CPU).
- **Tiền xử lý dữ liệu:** Đọc file CSV, tách đặc trưng X và nhãn y (MPG), chia các tập train/validation/test theo tỉ lệ 7:2:1, chuẩn hóa đặc trưng bằng StandardScaler (fit trên train, transform trên cả ba tập), chuyển đổi sang PyTorch tensor.
- **Xây dựng DataLoader:** Tạo class CustomDataset kế thừa `torch.utils.data.Dataset` với các phương thức `__len__` và `__getitem__`, khởi tạo DataLoader cho train (shuffle=True) và validation (shuffle=False).
- **Kiến trúc mô hình:** Xây dựng class MLP kế thừa `nn.Module` với 2 lớp ẩn, mỗi lớp theo sau bởi ReLU activation, lớp output không có activation (regression), sử dụng `squeeze(1)` để đảm bảo output shape phù hợp.
- **Loss và Optimizer:** Khai báo `nn.MSELoss()` và `torch.optim.SGD()`, cài đặt learning rate phù hợp.
- **Hàm đánh giá:** Cài đặt hàm `r_squared` để tính R^2 score: $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$ với $SS_{res} = \sum(y_{true} - y_{pred})^2$ và $SS_{tot} = \sum(y_{true} - \bar{y})^2$.
- **Vòng lặp huấn luyện:** Chuyển đổi giữa `model.train()` và `model.eval()`, thực hiện forward-backward-update cho mỗi batch, tính và lưu train/validation loss và R^2 score mỗi epoch, sử dụng `torch.no_grad()` khi đánh giá.
- **Đánh giá cuối cùng:** Tính R^2 score trên tập test sau khi huấn luyện hoàn tất.

Bảng cấu hình siêu tham số – Bài 1 (Auto MPG)

Tham số	Giá trị
Learning rate (`lr`)	0.01
Số epoch (`epochs`)	100
Batch size (`batch_size`)	32
Hidden dimensions (`hidden_dims`)	64
Val/Test split	0.2 / 0.125
Seed cố định (`random_state`)	59
Loss function	MSELoss
Optimizer	SGD
Evaluation metric	R^2 score



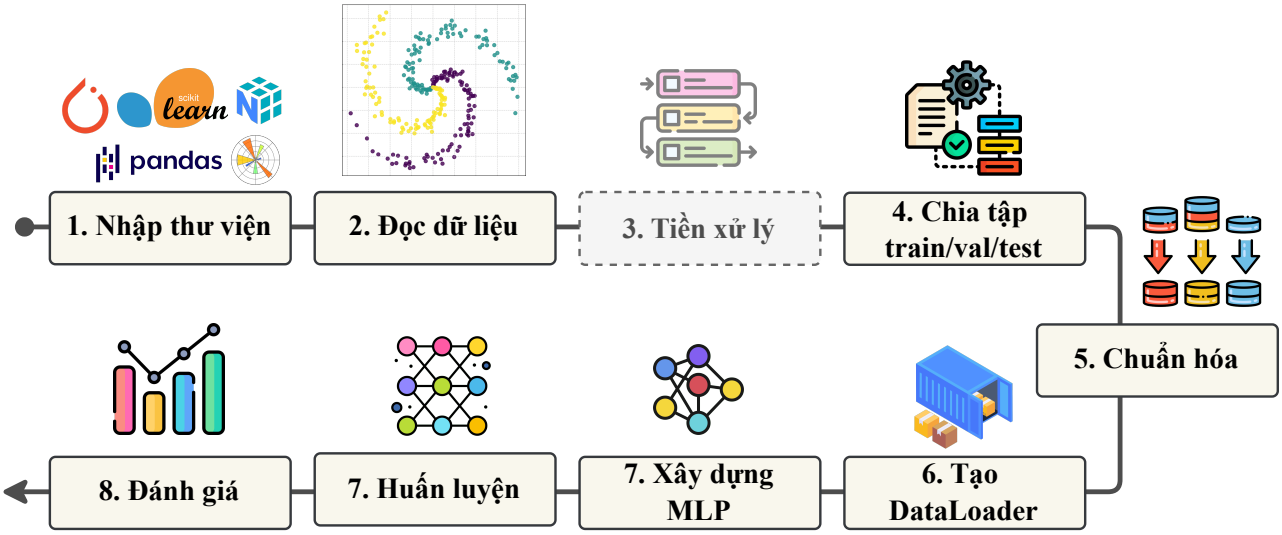
Hình 3: Trực quan hóa kết quả huấn luyện training/validation loss và accuracy qua các epoch, cho thấy sự hội tụ ổn định của mô hình.

II.2. Bài tập 2: Phân loại với dữ liệu phi tuyến

II.2.1. Phát biểu bài toán

Nội dung

Xây dựng mạng **Multi-layer Perceptron (MLP)** với PyTorch cho bài toán **Nonlinear Data Classification**. Thực hiện tiền xử lý, tách tập train/validation/test, xây dựng DataLoader, huấn luyện bằng SGD với hàm loss Cross Entropy, theo dõi loss/accuracy và đánh giá cuối cùng trên tập test.



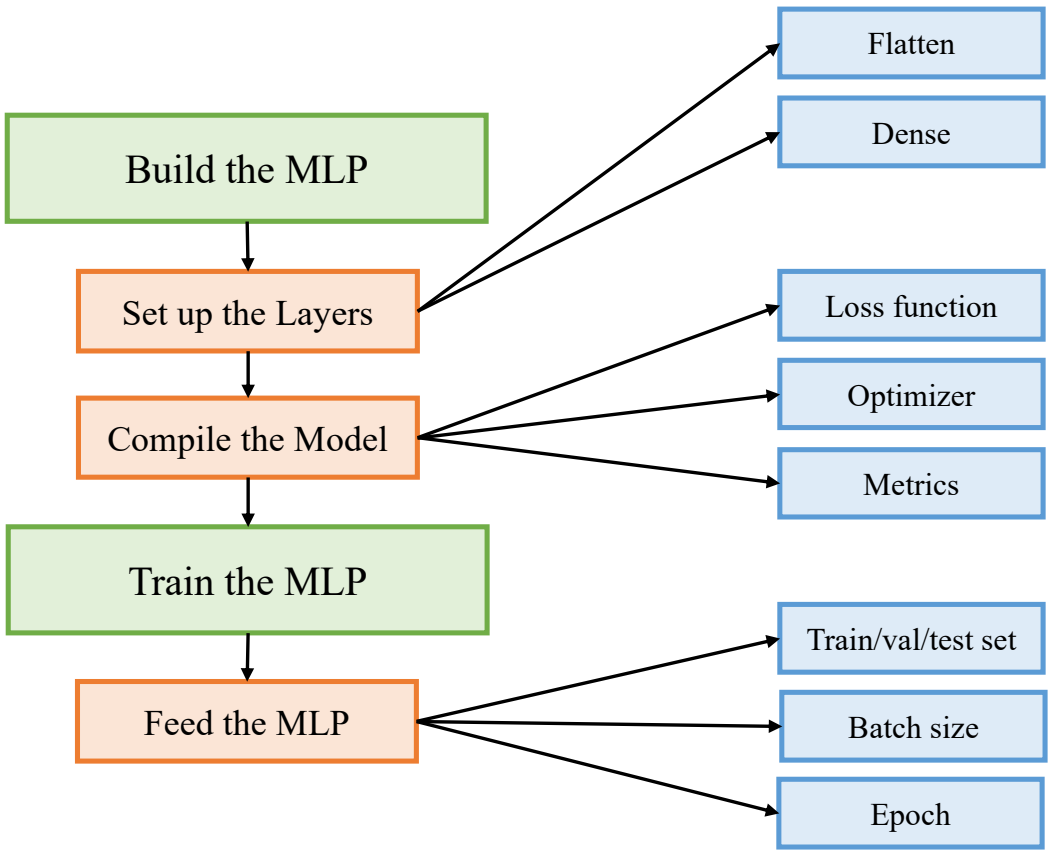
Hình 4: Quy trình từng bước cho bài tập 2.

Trong bài tập này, ta làm quen với khả năng của mạng neural network trong việc học các ranh giới quyết định phi tuyến (nonlinear decision boundaries) thông qua bài toán phân loại đa lớp. Bộ dữ liệu gồm 300 mẫu trong không gian 2D được chia thành 3 lớp với phân bố phi tuyến phức tạp. Mô hình MLP gồm 1 lớp ẩn với hàm kích hoạt ReLU, được tối ưu hóa bằng thuật toán Stochastic Gradient Descent với hàm loss Cross Entropy. Quá trình huấn luyện được giám sát qua hai chỉ số: Cross Entropy loss và classification accuracy.

II.2.2. Mục tiêu của bài tập

- Hiểu rõ khả năng của MLP trong việc học các đặc trưng phi tuyến và ranh giới quyết định phức tạp.
- Thành thạo xây dựng mạng MLP cho bài toán phân loại đa lớp với PyTorch.
- Làm việc với dữ liệu dạng `.npy` (NumPy Array File), xử lý dictionary chứa đặc trưng và nhãn.
- Hiểu và áp dụng hàm loss Cross Entropy cho bài toán phân loại đa lớp, nắm vững công thức tính loss trên batch.

- Cài đặt hàm `compute_accuracy()` để đánh giá hiệu suất phân loại.
- Tự động xác định số lớp đầu ra (output_dims) bằng `torch.unique` để mô hình thích ứng với các bài toán khác nhau.
- Thu thập và trực quan hóa đường hội tụ loss/accuracy trên train và validation qua các epoch.
- Hiểu sự khác biệt trong cấu hình mô hình giữa bài regression và classification (số lớp ẩn, activation cuối, hàm loss).



Hình 5: Minh họa các bước xây dựng và huấn luyện mô hình trong PyTorch.

II.2.3. Yêu cầu

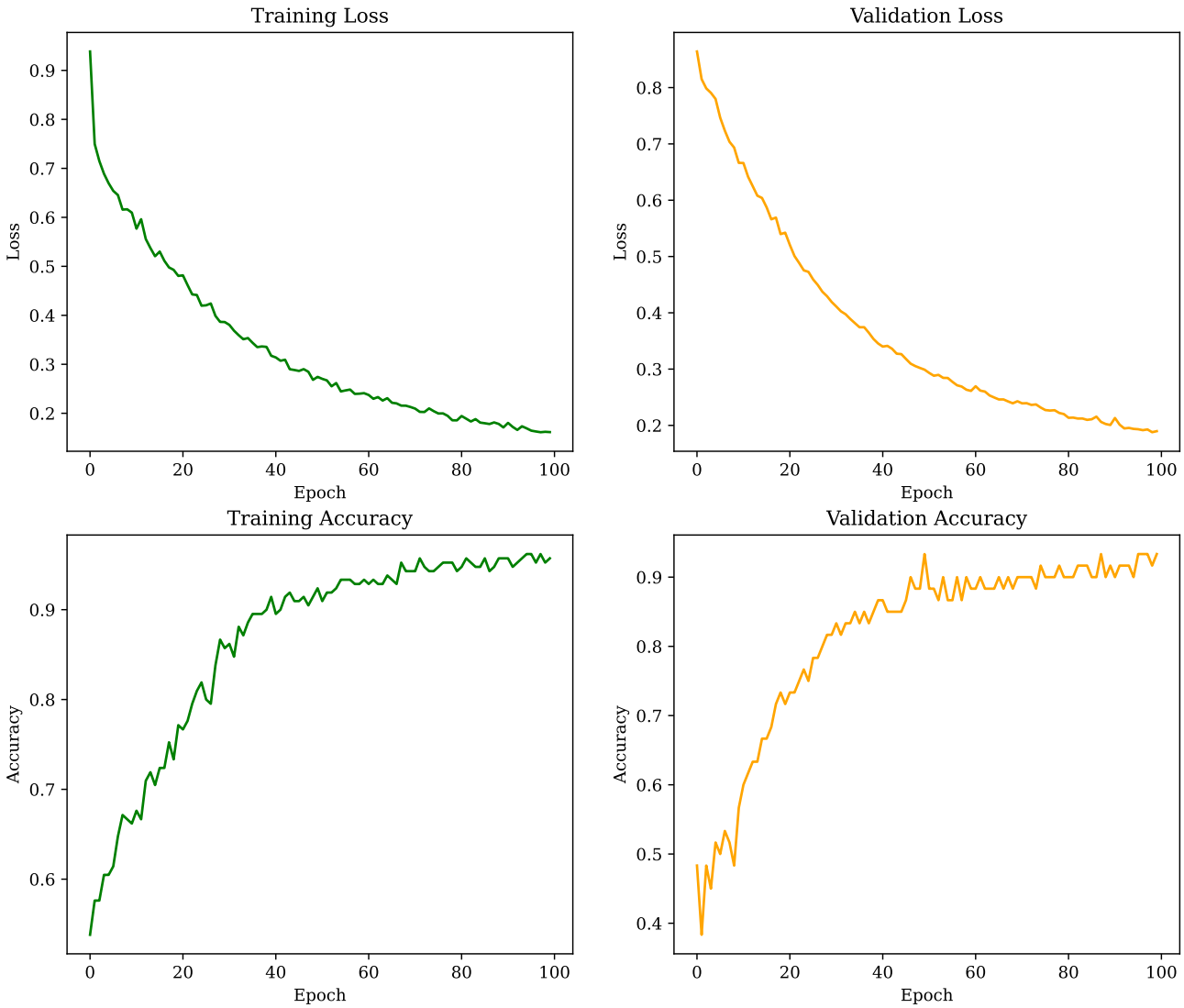
Dựa vào hướng dẫn chi tiết dưới đây, hãy hoàn thiện pipeline huấn luyện mô hình MLP cho bài toán phân loại phi tuyến:

- **Chuẩn bị môi trường:** Tải bộ dữ liệu NonLinear_data.npy bằng gdown hoặc thủ công, import các thư viện PyTorch, NumPy, matplotlib, scikit-learn, cài đặt seed cố định và thiết bị GPU/CPU.
- **Đọc và khám phá dữ liệu:** Sử dụng `np.load()` với `allow_pickle=True` để đọc file .npy, trích xuất `X` (ma trận 300×2) và `y` (vector 300 nhãn) từ dictionary, kiểm tra shape của dữ liệu.

- **Tiền xử lý dữ liệu:** Chia train/validation/test theo tỉ lệ 7:2:1, chuẩn hóa đặc trưng bằng StandardScaler (fit trên train, transform trên cả ba tập), chuyển đổi sang PyTorch tensor với `dtype=torch.float32` cho X và `dtype=torch.long` cho y (quan trọng cho CrossEntropyLoss).
- **Xây dựng DataLoader:** Tạo class CustomDataset cho cả train, validation và test, khởi tạo DataLoader với batch_size=32, train có shuffle=True, còn val và test có shuffle=False.
- **Kiến trúc mô hình:** Xây dựng class MLP với 1 lớp ẩn (không dùng `squeeze(1)` ở output vì là bài phân loại đa lớp), tự động xác định `output_dims` bằng `torch.unique(y_train).shape[0]`, đặt `hidden_dims=128` để có đủ capacity học đặc trưng phi tuyến phức tạp.
- **Loss và Optimizer:** Khai báo `nn.CrossEntropyLoss()` (đã tích hợp softmax, không cần thêm activation ở output layer) và `torch.optim.SGD()`, hiểu rõ công thức Cross Entropy trên batch: $\text{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$.
- **Hàm đánh giá:** Cài đặt `compute_accuracy` sử dụng `torch.max()` để lấy class có xác suất cao nhất, so sánh với nhãn thực tế và tính tỉ lệ dự đoán đúng.
- **Vòng lặp huấn luyện:** Thu thập predictions và targets từ tất cả batches (sử dụng `.detach().cpu()`, `torch.cat()`), tính accuracy sau khi hoàn tất mỗi epoch trên toàn bộ tập train và validation, lưu lịch sử train/val loss và accuracy.
- **Trực quan hóa:** Vẽ 4 subplot (2×2) hiển thị: Training Loss, Validation Loss, Training Accuracy, Validation Accuracy theo epoch, sử dụng màu xanh cho train và cam cho validation.
- **Đánh giá cuối cùng:** Tính accuracy trên tập test bằng cách thu thập predictions từ test_loader, sử dụng `torch.no_grad()` và `model.eval()`.

Bảng cấu hình siêu tham số – Bài 2 (Nonlinear Classification)

Tham số	Giá trị
Learning rate (`lr`)	0.1
Số epoch (`epochs`)	100
Batch size (`batch_size`)	32
Hidden dimensions (`hidden_dims`)	128
Input dimensions (`input_dims`)	2
Output dimensions (`output_dims`)	3 (auto-detect)
Val/Test split	0.2 / 0.125
Seed cố định (`random_state`)	59
Loss function	CrossEntropyLoss
Optimizer	SGD
Evaluation metric	Accuracy



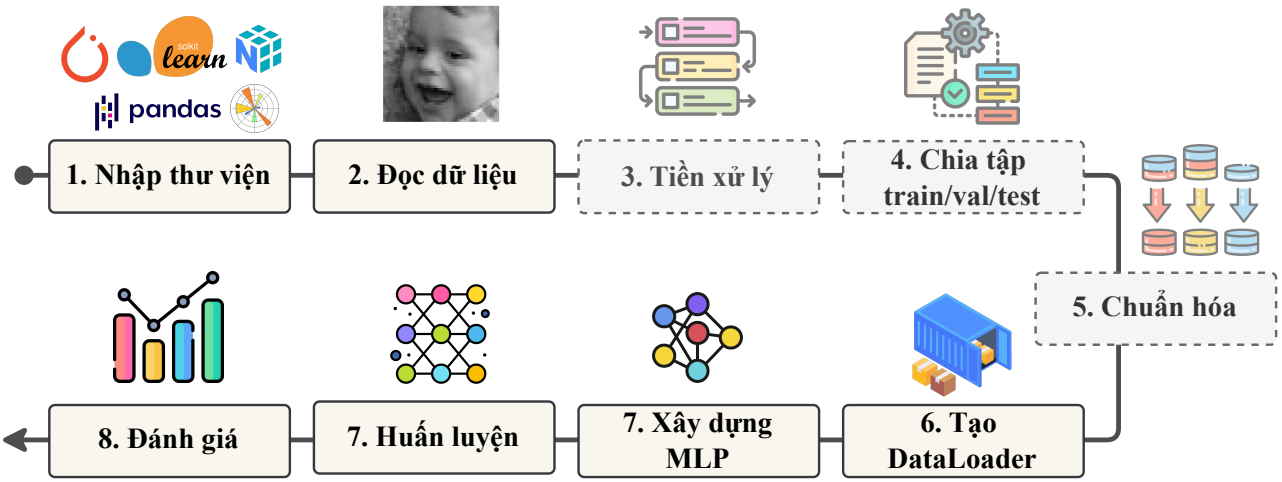
Hình 6: Trực quan hóa kết quả huấn luyện training/validation loss và accuracy qua các epoch, cho thấy sự hội tụ ổn định của mô hình.

II.3. Bài tập 3: Phân loại cảm xúc trên ảnh

II.3.1. Phát biểu bài toán

Nội dung

Xây dựng mạng **Multi-layer Perceptron (MLP)** với PyTorch cho bài toán **Facial Expression Recognition (FER-2013)**. Thực hiện tiền xử lý ảnh, xây dựng ImageDataset tùy chỉnh, tách tập train/validation/test theo cấu trúc thư mục, huấn luyện mạng sâu 3 lớp ẩn bằng SGD với hàm loss Cross Entropy, theo dõi loss/accuracy và đánh giá cuối cùng trên tập test.



Hình 7: Quy trình từng bước cho bài tập 3.

Trong bài tập này, ta mở rộng khả năng của MLP sang lĩnh vực Computer Vision thông qua bài toán phân loại cảm xúc khuôn mặt từ ảnh grayscale. Bộ dữ liệu FER-2013 chứa hàng nghìn ảnh khuôn mặt được tổ chức theo cấu trúc thư mục, mỗi thư mục con tương ứng với một loại cảm xúc. Mô hình MLP được thiết kế sâu hơn với 3 lớp ẩn (kiến trúc pyramid: 256, 128, 64 nodes) và hàm kích hoạt ReLU, nhằm trích xuất các đặc trưng phức tạp từ không gian pixel. Dữ liệu ảnh được chuẩn hóa về khoảng [-1, 1] và flatten trước khi đưa vào mạng. Bài tập này giúp hiểu rõ cách xử lý dữ liệu ảnh trong PyTorch và xây dựng pipeline hoàn chỉnh cho bài toán phân loại đa lớp trên dữ liệu thực tế.



Hình 8: Minh họa một số hình ảnh từ tập huấn luyện với các nhãn cảm xúc tương ứng (angry, happy, sad, surprise, etc.).

II.3.2. Mục tiêu của bài tập

- Làm quen với việc xử lý dữ liệu ảnh trong PyTorch: đọc file .zip, giải nén, quản lý cấu trúc thư mục.
- Xây dựng class `ImageDataset` tùy chỉnh kế thừa `torch.utils.data.Dataset` cho dữ liệu ảnh được tổ chức theo thư mục.
- Sử dụng `torchvision.io.read_image` và `torchvision.transforms.Resize` để đọc và resize ảnh.
- Hiểu và áp dụng kỹ thuật chuẩn hóa ảnh: `(img/127.5) - 1` để đưa pixel về khoảng [-1, 1].
- Tự động tạo mapping giữa tên lớp và index (`label2idx`, `idx2label`) từ cấu trúc thư mục.
- Tách train/validation theo tỉ lệ với `stratify` để đảm bảo phân bố cân bằng các lớp.
- Xây dựng mạng MLP sâu với kiến trúc pyramid (3 lớp ẩn) để xử lý dữ liệu ảnh phức tạp.
- Sử dụng `nn.Flatten()` để chuyển tensor ảnh 2D thành vector 1D trước khi đưa vào fully connected layers.
- Trực quan hóa batch dữ liệu ảnh với `matplotlib` để kiểm tra tính đúng đắn của pipeline.
- Huấn luyện mô hình với batch size lớn hơn (256) và theo dõi hội tụ trên dữ liệu thực tế quy mô lớn.

II.3.3. Yêu cầu

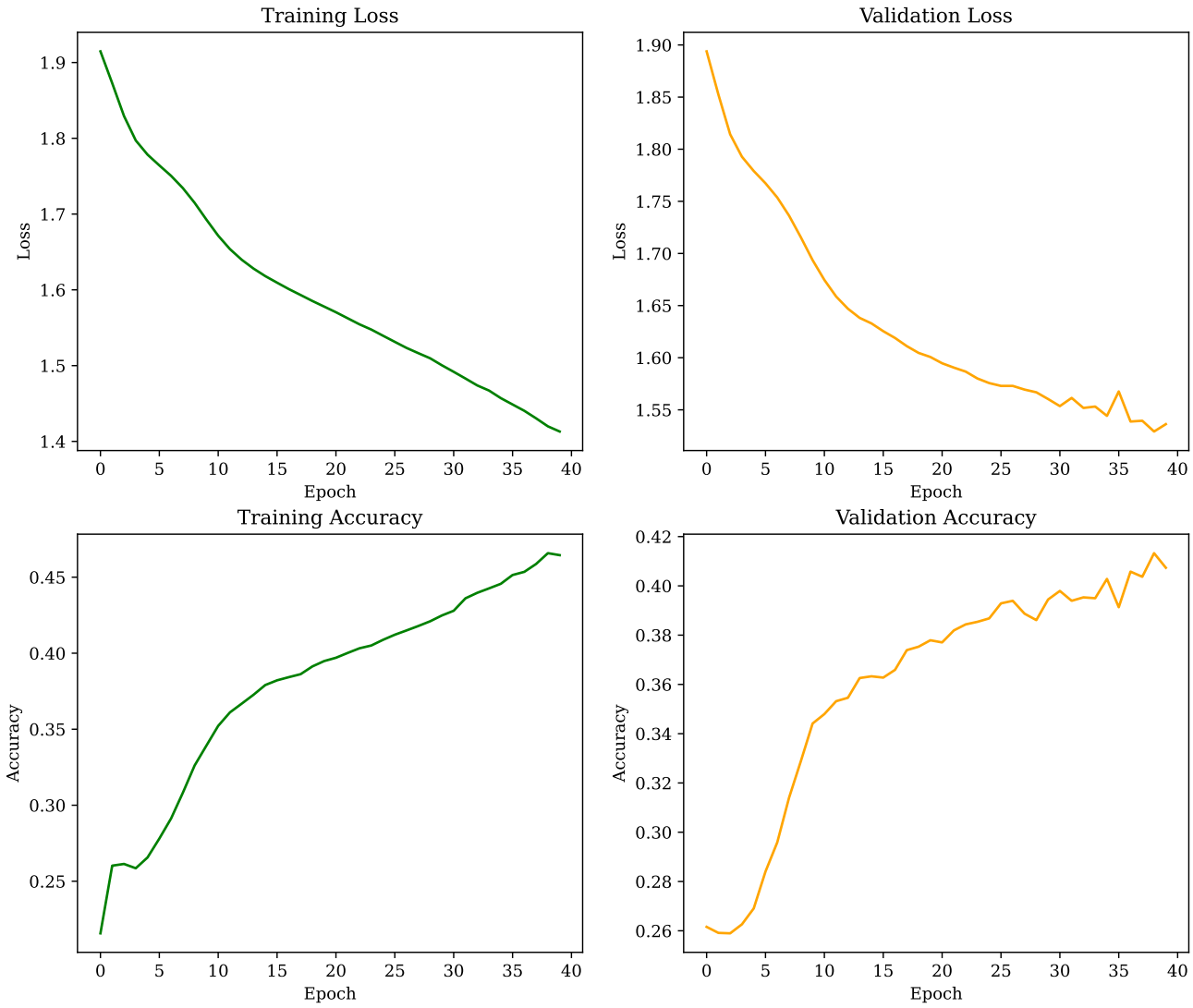
Dựa vào hướng dẫn chi tiết dưới đây, hãy hoàn thiện pipeline huấn luyện mô hình MLP cho bài toán phân loại cảm xúc trên ảnh:

- **Chuẩn bị dữ liệu:** Tải file FER-2013.zip bằng gdown, giải nén bằng lệnh `!unzip -q`, kiểm tra cấu trúc thư mục train/test và các thư mục con (angry, happy, sad, etc.).
- **Import thư viện:** Ngoài PyTorch, NumPy, pandas, cần thêm `cv2`, `torchvision` để xử lý ảnh.
- **Thiết lập môi trường:** Cài đặt seed, device, và kích thước ảnh chuẩn (`img_height=128`, `img_width=128`).
- **Tạo label mapping:** Sử dụng `os.listdir()` để lấy danh sách các lớp cảm xúc, tạo dictionary `label2idx` (mapping tên sang index) và `idx2label` (mapping index sang tên) cho việc encoding/decoding nhãn.
- **Xây dựng ImageDataset:** Tạo class kế thừa Dataset với các chức năng:
  - `read_img_files()`: Duyệt qua cấu trúc thư mục, thu thập đường dẫn ảnh và nhãn.
  - `__init__()`: Thực hiện `train_test_split` với `stratify=self.img_labels` để chia train/val cân bằng (80/20).
  - `__getitem__()`: Đọc ảnh bằng `read_image()`, resize về 128×128, chuyển sang `float32-`, áp dụng normalization `(img/127.5)-1` nếu `norm=True`, trả về tuple (img, label).
  - `__len__()`: Trả về số lượng ảnh.
- **Tạo DataLoader:** Khởi tạo DataLoader cho train (shuffle=True), validation (shuffle=False), và test (shuffle=False) với `batch_size=256`.
- **Trực quan hóa dữ liệu:** Sử dụng `next(iter(train_loader))` để lấy một batch, vẽ 9 ảnh đầu tiên (subplot 3×3) với `plt.imshow()` và `cmap="gray"`, hiển thị nhãn cảm xúc tương ứng.
- **Kiến trúc MLP sâu:** Xây dựng mạng với 3 lớp ẩn theo kiến trúc pyramid:
  - Layer 1: `input_dims sang hidden_dims*4` (128×128 sang 256)
  - Layer 2: `hidden_dims*4 sang hidden_dims*2` (256 sang 128)
  - Layer 3: `hidden_dims*2 sang hidden_dims` (128 sang 64)
  - Output: `hidden_dims sang output_dims` (64 sang num_classes)
  - Sử dụng `nn.Flatten()` ở đầu forward pass để chuyển ảnh 2D thành vector 1D.
  - ReLU activation sau mỗi lớp ẩn, không có activation ở output (CrossEntropyLoss đã tích hợp softmax).
- **Loss và Optimizer:** Khai báo `nn.CrossEntropyLoss()` và `torch.optim.SGD()` với `lr-=0.01`.
- **Hàm đánh giá:** Sử dụng lại hàm `compute_accuracy` từ bài 2.
- **Vòng lặp huấn luyện:** Huấn luyện 40 epochs, thu thập predictions từ tất cả batches bằng `.detach().cpu()` và `torch.cat()`, tính loss/accuracy cho cả tập train và validation mỗi epoch.

- **Trực quan hóa kết quả:** Vẽ 4 subplot (2×2) cho Training/Validation Loss và Accuracy theo epoch.
- **Đánh giá test set:** Tính accuracy trên tập test sau khi huấn luyện hoàn tất.

Bảng cấu hình siêu tham số – Bài 3 (FER-2013)

Tham số	Giá trị
Learning rate (`lr`)	0.01
Số epoch (`epochs`)	40
Batch size (`batch_size`)	256
Hidden dimensions (`hidden_dims`)	64
Image size (`img_height` × `img_width`)	128 × 128
Input dimensions (`input_dims`)	16384 (128×128)
Output dimensions (`output_dims`)	7 (auto-detect)
Train/Val split	0.8 / 0.2
Normalization	(img/127.5) - 1
Seed cố định (`random_state`)	59
Loss function	CrossEntropyLoss
Optimizer	SGD
Evaluation metric	Accuracy
MLP architecture	3 hidden layers (256, 128, 64)



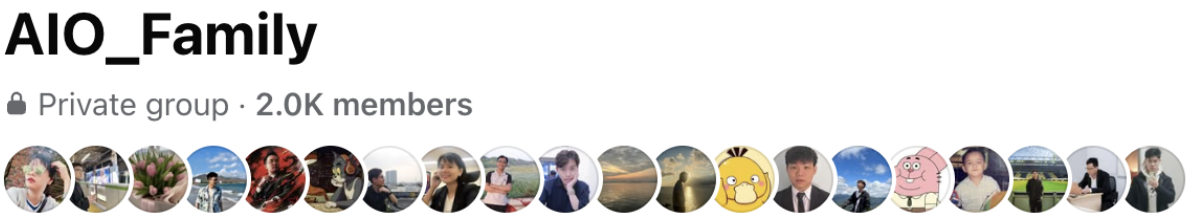
Hình 9: Trực quan hóa kết quả huấn luyện trainin/validation loss và accuracy qua 40 epochs, cho thấy khả năng của MLP trong việc học các đặc trưng cảm xúc từ ảnh khuôn mặt.

Phụ lục

1. **Dataset:** Các file dataset được đề cập trong bài có thể được tải tại đây.
2. **Hint và Solution:** Các file code gợi ý có thể được tải tại đây.
3. **Rubric:**

Mục	Kiến thức	Đánh giá
I.	- Kiến thức về việc cài đặt và huấn luyện mô hình Multi-layer Perceptron với PyTorch. - Kiến thức về các thành phần trong MLP (Input layer, Hidden layers, Output layer, Activation functions). - Kiến thức về Forward Propagation và Backpropagation. - Kiến thức về các phương pháp tiền xử lý dữ liệu dạng bảng, dữ liệu phi tuyến và dữ liệu ảnh. - Kiến thức về PyTorch Dataset, DataLoader và cách xây dựng pipeline huấn luyện hoàn chỉnh.	- Nắm được cách cài đặt mô hình MLP với PyTorch từ đầu. - Nắm được các thành phần và cơ chế hoạt động của MLP. - Hiểu được cách vận hành của Forward và Backward Propagation trong neural networks.
II.	- Các kiến thức và nội dung cơ bản về Multi-layer Perceptron.	- Nắm được kiến thức cơ bản về MLP và ứng dụng cho các bài toán regression, classification.

4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



Hình 10: Hình ảnh group Facebook AIO Q&A.